

NODE.JS

Architecture

Deep Dive — Beginner to Advanced

A complete, structured reference to Node.js internals — the Event Loop, V8 engine, libuv, async I/O, streams, clustering, and production patterns. Written for engineers who want to understand *why* Node.js works the way it does, not just how to use it.

Event-Driven	Non-Blocking I/O	V8 + libuv	Single-Threaded	Highly Scalable
--------------	------------------	------------	-----------------	-----------------

19 chapters · Internals, patterns & production tips

What is Node.js?

A JavaScript runtime built on Chrome's V8 engine — bringing JS to the server side.

Node.js was created by Ryan Dahl in 2009. He took Google's open-source V8 JavaScript engine — the same engine that powers Chrome — and wrapped it in a C++ program that can run outside the browser. The result: JavaScript on servers, running fast, async, and at scale.

Before Node, JavaScript was strictly a browser language. Server-side work required Python, Ruby, Java, or PHP. Node changed this by enabling a unified JS stack — the same language on client and server — while introducing a radically different concurrency model: the event loop.

CORE COMPONENTS

V8 Engine	Google's JS → machine code compiler. JIT-compiles hot functions for near-native speed.
libuv	C++ library providing: async I/O, a thread pool, DNS, timers, and the event loop itself.
npm	Node Package Manager — over 2 million open-source packages available instantly.
Core Modules	Built-in: fs, http, path, crypto, stream, cluster, os — no install required.

WHERE NODE.JS EXCELS

IDEAL USE CASES

- REST & GraphQL APIs — non-blocking DB queries handle thousands of concurrent requests
- Real-time apps — WebSockets, chat, live dashboards, collaborative tools
- Microservices — lightweight, fast startup, easy containerisation with Docker
- CLI tools — npm scripts, build tools, code generators (e.g. Vite, ESLint, TypeScript)
- Streaming servers — video, audio, file downloads using Node Streams
- BFF (Backend for Frontend) — proxy/aggregate multiple services for a UI layer

WHEN NOT TO USE NODE.JS

- CPU-intensive computation — video encoding, ML training, heavy image processing
- Pure mathematical workloads — scientific simulations, number crunching
- Multi-threaded database servers — PostgreSQL, MySQL are better suited here
(Workaround: use Worker Threads for CPU tasks if you must stay in Node)

Used in production by: Netflix, Uber, LinkedIn, NASA, PayPal, Walmart, and Airbnb.

The C10K Problem

Why traditional thread-per-request servers fail at scale — and how Node solves it.

In 1999, engineer Dan Kegel posed the 'C10K problem': can a single server handle 10,000 concurrent connections? Traditional web servers like Apache used one OS thread per request. Each thread consumes ~1–8 MB of RAM and has overhead from context switching.

10,000 users = 10,000 threads = 10+ GB RAM — just sitting idle, mostly waiting for database responses or file reads. The CPU is barely used; RAM is exhausted. Scaling means buying more hardware.

THREAD-PER-REQUEST MODEL (TRADITIONAL)

Problem	Detail
Memory waste	Each thread needs 1–8 MB stack. 10K threads = 10–80 GB RAM.
Context switching	OS switches between threads ~1,000× per second — CPU overhead.
Idle blocking	Threads wait idle on DB/file I/O. CPU has nothing to compute.
Slow startup	Spawning OS threads is expensive — adds latency per new request.

NODE'S ANSWER

Node uses **one main thread** with a non-blocking event loop. When an I/O operation is triggered (DB query, file read, HTTP request), Node hands it off to the OS or libuv's thread pool and immediately moves on to handle the next request. When the operation completes, the OS notifies Node via a callback.

THE NUMBERS

Traditional server: 10,000 users → ~10 GB RAM

Node.js: 10,000 users → ~50 MB RAM (200× less)

Node never sits idle — it is always processing the next queued callback

The V8 Engine

How Google's open-source JavaScript engine turns your code into machine instructions.

V8 is written in C++ and is the engine inside both Google Chrome and Node.js. It does not interpret JavaScript line-by-line like older engines — it compiles JS directly to native machine code using **JIT (Just-In-Time) compilation**.

COMPILATION PIPELINE

- | | |
|----------------|---|
| 1. Parsing | Source code → AST (Abstract Syntax Tree). Checks syntax, builds a tree of tokens. |
| 2. Ignition | Bytecode interpreter. Runs code quickly on first execution with moderate speed. |
| 3. TurboFan | Optimising JIT compiler. Detects 'hot' functions (called often) and compiles to native C++. |
| 4. De-optimize | If assumptions TurboFan made are violated (e.g. type changes), it reverts to bytecode. |

JIT IN PRACTICE

INTERPRETED (SLOW)

- First run — code interpreted
- ~100ms per operation
- No native code emitted
- Full flexibility, no assumptions

JIT COMPILED (FAST)

- Hot path — compiled to native
- ~1ms per operation (100× faster)
- CPU runs instructions directly
- Type-stable code stays compiled

V8 also manages the **heap** (where objects live) and runs **garbage collection** automatically using a generational GC — short-lived objects (most) are collected cheaply in the 'young generation', long-lived objects promoted to 'old generation'.

PERFORMANCE TIPS FOR V8

- Keep object shapes consistent — don't add/delete properties dynamically in hot loops
- Avoid mixing types in arrays (monomorphic arrays compile better than polymorphic)
- Use `--prof` flag to generate a V8 profiler log and identify hot paths
- Avoid giant closures — they pin memory in the old generation longer

The Event Loop

The heart of Node.js — a six-phase cycle that never stops.

The event loop is what allows Node.js to perform non-blocking I/O despite running JavaScript on a single thread. After your synchronous code runs and the call stack is empty, the event loop takes over — picking up and executing callbacks in a strict, repeating order across six phases.

THE SIX PHASES

Phase	Name	What Runs Here
①	Timers	setTimeout() and setInterval() callbacks whose delay has expired.
②	Pending I/O	I/O callbacks deferred from the previous loop iteration (e.g. TCP errors).
③	Idle / Prepare	Internal Node.js housekeeping only. Not user-visible.
④	Poll	Retrieves new I/O events. This is where Node waits if the queue is empty.
⑤	Check	setImmediate() callbacks — always runs after Poll phase.
⑥	Close	Close callbacks: socket.on('close'), server.on('close'), etc.

BETWEEN PHASES: MICROTASKS

Before the event loop transitions from one phase to the next, it fully drains the **microtask queue**. This includes process.nextTick() callbacks (highest priority) and resolved Promise.then() / queueMicrotask() callbacks. A flooding microtask queue will starve all I/O phases.

POLL PHASE — THE SMART WAITER

The Poll phase is special. If the poll queue is empty and no timers are scheduled, Node will block here — efficiently waiting for new I/O events from the OS rather than spinning in a busy loop. When a timer's threshold is reached, it moves on. This is how Node achieves near-zero CPU usage while idle.

The Call Stack

LIFO execution tracking — and why you must never block it.

The call stack is a simple data structure (Last In, First Out) that tracks the current execution context. Every function call pushes a frame; every return pops one. The event loop only picks up callbacks when the call stack is **completely empty**.

```
function add(a, b) { return a + b; }
function calculate() { return add(3, 4); }
calculate();

// Stack state during execution:
// [main()]
// [main(), calculate()]
// [main(), calculate(), add(3,4)] ← executing
// [main(), calculate()]          ← add returned 7
// [main()]                      ← calculate returned 7
// []                            ← event loop can now fire callbacks
```

THE BLOCKING PROBLEM

If synchronous code runs for a long time — a heavy loop, a recursive algorithm, a `readFileSync()` call — the call stack remains occupied. The event loop cannot run any callbacks. From every other user's perspective, **the server is frozen**.

NEVER DO THIS

- `fs.readFileSync()` in a handler
- `JSON.parse(huge_string)` on main
- Sorting 1M+ items synchronously
- `bcrypt` hashing on main thread
- `while(true)` busy wait loops

DO THIS INSTEAD

- `fs.readFile()` with callback/await
- Parse JSON in Worker Thread
- Sort in Worker Thread or offload
- Use async `bcrypt.hash()`
- `setTimeout` or `setImmediate`

RULE

- Any sync operation > ~1ms is dangerous on the main thread in production.
- Use Worker Threads (chapter 12) for CPU-bound work.
- Use async variants of every I/O operation — always.

libuv — The Async Engine

The C++ library that makes non-blocking I/O possible across all platforms.

libuv was written specifically for Node.js (though it is now used by other runtimes too). V8 runs your JavaScript. libuv handles *everything asynchronous*: the event loop, file I/O, networking, timers, DNS, and the thread pool.

Crucially, libuv abstracts over OS-specific async APIs so Node.js code runs identically on Linux, macOS, and Windows:

PLATFORM ABSTRACTION

Linux	epoll — highly scalable event notification, handles millions of file descriptors.
macOS	kqueue — BSD-derived event queue, similar capabilities to epoll.
Windows	IOCP (I/O Completion Ports) — Windows native async I/O model.

LIBUV RESPONSIBILITIES

Event Loop	The 6-phase loop. libuv owns this — it drives all callback execution.
Thread Pool	4 worker threads (default) for operations the OS can't do async natively.
Async File I/O	File reads/writes dispatched to thread pool so main thread never blocks.
Network I/O	TCP/UDP sockets handled via OS async APIs — fully non-blocking.
DNS	dns.lookup() uses thread pool. dns.resolve() uses c-ares (true async).
Timers	setTimeout / setInterval — implemented in libuv's timer heap.
Child Processes	spawn, exec — managed by libuv with IPC channels.

THE STACK

```

Your JavaScript Code
    ↓
V8 Engine (executes JS, manages heap/GC)
    ↓
Node.js Bindings (C++ bridge: fs, http, crypto...)
    ↓
libuv (event loop + thread pool + async I/O)
    ↓
OS Kernel (Linux epoll / macOS kqueue / Windows IOCP)
  
```

The Thread Pool

libuv's 4 background workers — handling what the OS can't do asynchronously.

Not every operation can be made truly async at the OS level. File system operations, cryptographic hashing, and DNS lookups via `dns.lookup()` are examples that would block if run on the main thread. libuv routes these to a pool of background worker threads so the main thread stays free.

WHAT USES THE THREAD POOL

File System	<code>fs.readFile</code> , <code>fs.writeFile</code> , <code>fs.stat</code> , <code>fs.open</code> — most fs operations.
Crypto	<code>crypto.pbkdf2</code> , <code>crypto.scrypt</code> , <code>crypto.randomBytes</code> , bcrypt hashing.
Compression	<code>zlib.gzip</code> , <code>zlib.deflate</code> , <code>zlib.inflate</code> — runs in background thread.
DNS Lookup	<code>dns.lookup()</code> (<code>getaddrinfo</code>) — blocking C function, thread pool offloads it.
Custom C++	Native addons can schedule their own work on the thread pool.

THREAD POOL SIZE

By default there are **4 worker threads**. If you run 5 concurrent `crypto.pbkdf2()` calls, the 5th queues until a thread is free. You can increase this — up to a maximum of 1024 — with an environment variable:

```
UV_THREADPOOL_SIZE=16 node server.js

# Or in code before any async I/O:
process.env.UV_THREADPOOL_SIZE = '16';
```

TUNING THE THREAD POOL

Default (4) is fine for most APIs — I/O is usually OS-async, not thread pool.
 Increase if you do heavy concurrent crypto (e.g. bcrypt on login endpoint).
 Set to `number_of_CPU_cores` for CPU-bound thread pool work.
 Monitor with: `async_hooks` or `clinic.js` to detect thread pool saturation.

Blocking vs Non-Blocking I/O

The fundamental concept that defines Node.js performance.

Every I/O operation in Node.js has two flavours: blocking (sync) and non-blocking (async). Blocking calls halt the entire thread until the operation completes. Non-blocking calls register a callback and immediately return — the event loop is notified when the result is ready.

BLOCKING (SYNC)

- Halts the main thread entirely
- All other requests wait
- CPU sits idle during wait
- Simple to read, dangerous to use
- Only safe in startup scripts / CLI

NON-BLOCKING (ASYNC)

- Registers callback, returns immediately
- Other requests continue processing
- CPU stays busy with next callback
- Scales to 10,000+ concurrent users
- Required in all production server code

SIDE-BY-SIDE CODE COMPARISON

```
// BLOCKING – freezes server for ALL users while file loads
const data = fs.readFileSync('/large/file.json');
res.json(JSON.parse(data)); // Only reached after full read

// NON-BLOCKING – returns immediately, callback fires when ready
fs.readFile('/large/file.json', (err, data) => {
  if (err) return next(err);
  res.json(JSON.parse(data)); // Fires when OS is done
});
// ← Event loop continues here immediately, serving other requests

// MODERN – async/await (syntactic sugar over Promises)
const data = await fs.promises.readFile('/large/file.json');
res.json(JSON.parse(data));
```

THE GOLDEN RULE

Never use any `*Sync()` method inside a request handler on a production server.
Exception: module-level startup code that runs once before the server opens.
`fs.readFileSync` at module load time is acceptable (e.g. loading config).

Microtasks vs Macrotasks

Callback priority — not all callbacks are created equal.

Node.js uses a strict priority hierarchy to decide which callback runs next. Understanding this order prevents subtle bugs where timers fire unexpectedly or Promises resolve in a different order than expected.

PRIORITY ORDER (HIGHEST → LOWEST)

Priority	API	Queue	Notes
1 — Highest	<code>process.nextTick()</code>	nextTick Queue	Runs before ANYTHING else after current op. Can starve I/O if overused.
2	<code>Promise.then/.catch</code>	Microtask Queue	Runs after nextTick. All microtasks drain before next event loop phase.
2	<code>queueMicrotask()</code>	Microtask Queue	Same priority as Promise callbacks.
■■■ boundary ■■■			Microtasks above · Macrotasks below
3	<code>setImmediate()</code>	Check Phase	Fires after I/O callbacks in the check phase.
4	<code>setTimeout(fn, 0)</code>	Timer Phase	Fires after minimum ~1ms delay (not truly zero).
5 — Lowest	I/O Callbacks	Poll Phase	Network, file, DB results — main event source.

PRACTICAL EXAMPLE

```
setTimeout(() => console.log('A: setTimeout'), 0);
setImmediate(() => console.log('B: setImmediate'));
Promise.resolve().then(() => console.log('C: Promise'));
process.nextTick(() => console.log('D: nextTick'));
console.log('E: sync');
```

```
// Output order:
// E: sync           ← synchronous, call stack
// D: nextTick       ← microtask (highest priority)
// C: Promise        ← microtask
// A: setTimeout     ← macrotask (timer phase)
// B: setImmediate   ← macrotask (check phase)
```

A Request's Full Journey

Tracing one HTTP request from browser to database and back — step by step.

Understanding exactly what happens inside Node.js during a single API request helps you reason about performance, debug issues, and write better async code.

THE 8-STEP JOURNEY

Step	What Happens	Node.js Internals
1	Browser sends HTTP GET /api/users	TCP connection established, TLS handshake if HTTPS.
2	NGINX routes to Node process	Load balancer selects process (round-robin or least-conn).
3	http.Server callback fires	req/res objects created. Poll phase delivers I/O event to event loop.
4	Express matches route	Middleware chain runs: auth, logger, body parser — all synchronous or async.
5	await db.query(sql)	libuv sends TCP packet to DB. Promise created. Call stack clears.
6	Node is FREE — handles others	Event loop processes other requests, timers, callbacks. Zero idle.
7	DB responds → callback fires	Poll phase picks up OS notification → Promise resolves → microtask queue.
8	res.json(data) → response sent	Node writes to TCP socket. NGINX forwards. Browser receives JSON.

STEPS 5–7 ARE THE MAGIC

Between step 5 (query sent) and step 7 (result received), zero CPU is wasted.

Node is actively handling other requests, timers, and callbacks.

This is why Node handles 10,000 concurrent DB requests efficiently.

Traditional servers would have 10,000 threads sitting idle in this window.

Node.js Streams

Process data chunk by chunk — never load it all into memory at once.

Streams are one of Node's most powerful and underused features. Instead of loading an entire file or HTTP response into RAM, streams process data in small chunks (default 64 KB). This makes Node capable of serving multi-gigabyte files with minimal memory usage.

THE FOUR STREAM TYPES

Readable	Produces data. Examples: <code>fs.createReadStream()</code> , <code>http.IncomingMessage</code> , <code>process.stdin</code> .
Writable	Consumes data. Examples: <code>fs.createWriteStream()</code> , <code>http.ServerResponse</code> , <code>process.stdout</code> .
Duplex	Both readable and writable. Example: <code>net.Socket</code> — you can read from and write to it.
Transform	A duplex stream that modifies data as it passes through. Examples: <code>zlib.createGzip()</code> , <code>crypto.createHash()</code> .

PIPE — COMPOSING STREAMS

```
// Stream a 2 GB file with compression – uses only ~64 KB RAM
const fs = require('fs');
const zlib = require('zlib');

const readable = fs.createReadStream('large-video.mp4');
const gzip      = zlib.createGzip();
const writable  = fs.createWriteStream('large-video.mp4.gz');

readable.pipe(gzip).pipe(writable);

// HTTP streaming response
app.get('/video', (req, res) => {
  res.setHeader('Content-Type', 'video/mp4');
  fs.createReadStream('movie.mp4').pipe(res);
});
```

STREAMS VS BUFFERING

WITHOUT STREAMS (DANGEROUS)

- Load entire 2 GB file into RAM
- Server crashes — out of memory
- User waits for full file before any data
- 1 large file = server out of memory

WITH STREAMS (CORRECT)

- Only 64 KB in memory at any time
- Scales to any file size
- User receives first bytes immediately
- 100 concurrent streams = 6.4 MB RAM

STREAMS & BACKPRESSURE

Backpressure: when writable can't keep up with readable, `pipe()` pauses the source.

Without backpressure (manual pipe), buffers grow unbounded → memory leak.

Always use `pipe()` or the `stream/promises pipeline()` — never manually loop.

`pipeline()` from `stream/promises` also handles error propagation correctly.

Worker Threads

True multi-threading in Node.js for CPU-intensive workloads.

Worker Threads (introduced in Node 10, stable in Node 12) allow Node.js to run JavaScript in parallel threads — each with its own V8 instance, event loop, and memory heap. They communicate with the main thread via message passing, not shared memory (though `SharedArrayBuffer` is available for advanced use).

Workers are specifically for **CPU-bound** work. They do not help with I/O-bound tasks — for those, the event loop's async model is already optimal.

WHEN TO USE WORKER THREADS

USE WORKER THREADS FOR

- Image / video processing — resizing, transcoding, thumbnail generation
- Cryptographic operations — custom hashing, key derivation at high volume
- Large JSON parsing — parsing 10 MB+ JSON blocks the main thread
- Machine learning inference — running TensorFlow.js models
- Heavy data transformation — ETL pipelines, large array operations

BASIC USAGE

```
// worker.js
const { workerData, parentPort } = require('worker_threads');
const result = heavyComputation(workerData.input);
parentPort.postMessage({ result });

// main.js
const { Worker } = require('worker_threads');

function runWorker(input) {
  return new Promise((resolve, reject) => {
    const w = new Worker('./worker.js', { workerData: { input } });
    w.on('message', resolve);
    w.on('error', reject);
  });
}

const result = await runWorker(largeDataset);
```

WORKER THREAD VS CHILD PROCESS

Worker Threads: shared memory possible, lower overhead, same process group.

Child Process: separate OS process, fully isolated, use for untrusted code.

Thread Pool (libuv): automatic, for Node's built-in async ops (fs, crypto).

Scaling Node.js

From a single process to multi-core, multi-server, and cloud-scale deployment.

Node.js uses one CPU core per process. A modern server has 8, 16, or 32 cores — running a single Node process leaves nearly all of them idle. Scaling out means running multiple Node processes and routing traffic between them.

LAYER 1 — CLUSTER MODULE (SINGLE MACHINE)

Node's built-in cluster module forks one worker process per CPU core. The master process receives all incoming connections and distributes them round-robin to workers. Workers share the same TCP port.

```
const cluster = require('cluster');
const os = require('os');

if (cluster.isPrimary) {
  const numCPUs = os.cpus().length;
  for (let i = 0; i < numCPUs; i++) cluster.fork();
  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died. Reforking.`);
    cluster.fork();
  });
} else {
  require('./app'); // Each worker runs your Express app
}
```

LAYER 2 — LOAD BALANCER (MULTIPLE SERVERS)

NGINX or AWS ALB sits in front of multiple Node.js server instances. It handles SSL termination, health checks, and distributes requests. Each server can itself run a cluster of Node processes.

LAYER 3 — KUBERNETES (CLOUD SCALE)

Docker containers package your Node app. Kubernetes orchestrates them — auto-scaling based on CPU/memory metrics, health checks, rolling deploys, and service discovery. Each Pod runs one Node process (let K8s handle the scaling).

PM2	Process manager for production: auto-restart, log aggregation, cluster mode, monitoring
Redis Sessions	Stateless architecture — store sessions in Redis, not in-process memory, for horizontal scaling
Health Checks	Expose /health endpoint returning 200. Load balancers ping it; unhealthy pods are replaced
Graceful Shutdown	Handle SIGTERM: stop accepting new connections, finish in-flight requests, close DB connections

Error Handling

Async errors are silent killers — every async path needs explicit handling.

Unhandled errors in synchronous code throw exceptions that crash the process. Unhandled errors in asynchronous code silently disappear — in older Node versions. Since Node 15, an unhandled Promise rejection **terminates the process**. There is no excuse for missing error handlers.

THREE ASYNC PATTERNS AND THEIR ERROR HANDLING

```
// 1. Callbacks (legacy) – error is always the FIRST argument
fs.readFile(path, (err, data) => {
  if (err) return res.status(500).json({ error: err.message });
  res.json(JSON.parse(data));
});

// 2. Promises – always attach .catch()
fetch('/api/data')
  .then(r => r.json())
  .then(data => res.json(data))
  .catch(err => res.status(500).json({ error: err.message }));

// 3. async/await – cleanest, use try/catch
async function handler(req, res) {
  try {
    const data = await db.query('SELECT * FROM users');
    res.json(data);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
}
```

PROCESS-LEVEL SAFETY NETS

```
// Catch any synchronous throw not caught by try/catch
process.on('uncaughtException', (err) => {
  logger.error('Uncaught:', err);
  process.exit(1); // Always exit – state is now unknown
});

// Catch Promises with no .catch() handler
process.on('unhandledRejection', (reason, promise) => {
  logger.error('Unhandled Rejection:', reason);
  process.exit(1); // Node 15+ does this automatically
});
```

ERROR HANDLING RULES

Every async operation must have an error handler — no silent failures.

After `uncaughtException`, always exit — the process is in unknown state.

Use a central Express error middleware: `app.use((err, req, res, next) => ...)`

Log the full stack trace, not just `err.message` — you need the call location.

Production Architecture

How real companies deploy Node.js — the full stack from edge to database.

A production Node.js system is never just 'Node + a database'. It involves layers of infrastructure working together to provide reliability, performance, and observability at scale.

TYPICAL PRODUCTION STACK

Layer	Technology	Purpose
Edge / CDN	Cloudflare, Fastly	Static assets, DDoS protection, global edge caching, TLS termination.
Load Balancer	NGINX, AWS ALB	SSL offload, health checks, weighted routing, connection pooling.
App Servers	Node.js Cluster	Express/Fastify API, business logic. PM2 for process management.
Cache	Redis	Session storage, query result caching, pub/sub for real-time events.
Message Queue	Kafka / RabbitMQ	Async job processing: emails, notifications, report generation.
Database	PostgreSQL / MySQL	Durable relational storage with connection pooling (pg-pool).
Object Storage	S3, GCS	User uploads, static files, exports — never store files on app server.
Background Jobs	Bull / BullMQ	Scheduled tasks, retry logic, concurrency-controlled workers.
Observability	Datadog / Grafana	Metrics, distributed tracing, structured logs, alerting.

KEY PRODUCTION PRINCIPLES

- Stateless API servers — any instance can handle any request (enables auto-scaling)
- 12-Factor App methodology — config in env vars, logs to stdout, disposable processes
- Connection pooling — never open a new DB connection per request (pg-pool, mongoose pool)
- Circuit breakers — fail fast when downstream services are degraded (opossum library)
- Structured logging — JSON logs (pino, winston) queryable in Datadog / CloudWatch

Performance Tips

Practical patterns for writing Node.js that actually scales under load.

ALWAYS DO

- Use `async/await` everywhere
- Connection pooling (pg-pool etc.)
- Cache expensive queries in Redis
- Worker threads for CPU work
- Enable HTTP/2 + gzip/brotli
- Use streams for large data
- Profile with `--prof / clinic.js`
- Set `NODE_ENV=production`

NEVER DO

- Use `*Sync()` methods in handlers
- Open new DB conn per request
- Heavy computation on main thread
- Parse large JSON on main thread
- Store sessions in process memory
- Ignore error handling
- Log synchronously in hot paths
- Forget to close DB connections

PROFILING NODE.JS

--prof flag	Built-in V8 profiler. Run: <code>node --prof app.js</code> , then: <code>node --prof-process isolate-*.log</code>
clinic.js	<code>npm i -g clinic</code> — <code>clinic doctor</code> , <code>clinic flame</code> (flamegraph), <code>clinic bubbleprof</code> (async).
0x	<code>npm i -g 0x</code> — generates interactive flamegraph. Spot CPU hot paths instantly.
async_hooks	Low-level API to track async resource lifetimes — useful for finding orphaned ops.
pino	Fastest structured logger for Node. Use over <code>console.log</code> in production.

HTTP/2 & COMPRESSION

```
const compression = require('compression');
app.use(compression({ level: 6 })); // gzip – halves typical payload

// HTTP/2 via Node built-in (requires TLS)
const http2 = require('http2');
const server = http2.createSecureServer({ key, cert }, app);

// Or use NGINX as HTTP/2 terminator (recommended)
// NGINX → HTTP/2 → app servers → HTTP/1.1 internally
```

Interview Q&A;

The questions every Node.js interview covers — answered precisely.

What is the Event Loop?

A continuously running loop in libuv that picks up callbacks when the JS call stack is empty. It runs in 6 phases: timers → pending I/O → idle → poll → check → close. Between phases, microtasks (nextTick, Promises) are fully drained.

What is libuv and why does Node need it?

libuv is a C++ library providing Node's async I/O, the event loop, a 4-thread pool, DNS resolution, and timer implementation. V8 executes JavaScript; libuv handles everything async. Without libuv, every file read and DNS lookup would block the main thread.

How does Node handle 10,000 concurrent connections?

One main thread delegates I/O to the OS (epoll/kqueue/IOCP) or libuv's thread pool. Node is notified via callbacks when operations complete. No thread-per-request — 10,000 connections use ~50 MB RAM vs ~10 GB for Apache's thread-per-request model.

setImmediate() vs setTimeout(fn, 0) — what's the difference?

setTimeout fires in the timer phase after a minimum ~1ms delay. setImmediate fires in the check phase — always after I/O callbacks. Inside an I/O callback, setImmediate ALWAYS fires before setTimeout. Outside I/O, the order is non-deterministic (depends on OS timer precision).

process.nextTick() vs Promise.then()?

Both are microtasks. nextTick has higher priority — it runs before Promise.then(). Both drain completely before the event loop moves to the next phase. Abuse of nextTick (recursive nextTick) will starve all I/O and timer callbacks.

What is the difference between CPU-bound and I/O-bound?

I/O-bound: bottleneck is waiting for external systems (DB, disk, network). Node excels here — non-blocking I/O means zero idle time. CPU-bound: bottleneck is computation (image processing, encryption, sorting). Node's single thread is a weakness here — use Worker Threads to offload.

How do you prevent memory leaks in Node.js?

Common causes: global variables growing unbounded, event listeners not removed, closures holding references, caches without eviction policies. Detection: heapdump + Chrome DevTools, process.memoryUsage() monitoring, clinic.js, or --heap-prof flag. Always call emitter.removeListener() or use once().

The Full Cheat Sheet

Every critical Node.js concept condensed into one reference page.

Topic	Key Facts
V8 Engine	Google's JS→machine code compiler. JIT makes hot functions 100× faster than interpreted. Handles GC and memory.
Event Loop	6 phases: timers → pending I/O → idle → poll → check → close. Microtasks drain between every phase.
libuv	C++ async engine. Provides: event loop, thread pool (4 workers), async file I/O, network, DNS, timers.
Thread Pool	4 workers for: fs, crypto, zlib, dns.lookup. Set UV_THREADPOOL_SIZE=N to increase (max 1024).
Call Stack	LIFO. Event loop only fires callbacks when call stack is empty. Never block with sync work.
Async Patterns	callbacks → Promises → async/await. Always handle errors. Unhandled rejections exit in Node 15+.
Streams	4 types: Readable, Writable, Duplex, Transform. Use pipe(). Only 64 KB in RAM regardless of file size.
Worker Threads	True JS multi-threading. Use for CPU-bound work: image proc, heavy math, large JSON parse.
Cluster	1 worker per CPU core. Master distributes connections. PM2 manages and auto-restarts workers.
Scaling	Cluster (1 machine) → NGINX / ALB (multiple servers) → K8s (cloud). Stateless + Redis sessions.
Microtask Order	process.nextTick() > Promise.then() > queueMicrotask() > setImmediate() > setTimeout(fn,0)
Blocking I/O	Never use *Sync() in request handlers. 1 blocked call freezes all users. Always async.
Production	PM2 + cluster, Redis cache, NGINX reverse proxy, connection pools, structured logs, health endpoint.

Node.js is fast not because it avoids work — but because it never waits idle.

NODE.JS

Architecture Deep Dive

18 chapters covering V8, the Event Loop, libuv, the Thread Pool, Streams, Worker Threads, Clustering, Error Handling, Production Architecture, Performance, and Interview Preparation.

Chapters	18
Core Concepts	Event Loop · V8 · libuv · Streams · Workers
Focus	Internals, Patterns & Production